

sd05. Ingestion Pipeline (Metrics · \$12 Healthcare Records variant)

First-hand 1p3a P0 (bank #7 + #5): "Internal metric ingestion service — client agent logic, sharding, availability, decoupling write and query concerns, choice of infra" and "Healthcare data ingestion pipeline handling millions of records" (8 named discussion points). JD tie-in: OCI telemetry/control-plane pipelines; "operationally focused" — the DLQ/replay story is the senior signal.

Index

1. Crux & why hard
2. Clarifying questions
3. FR / NFR
4. Back-of-envelope
5. HLD
6. Client-side agent (key code)
7. Sharding & broker choice
8. Write path vs query path (decoupling)
9. Data model
10. Failure handling: DLQ + replay
11. Ops readiness
12. Healthcare records variant
13. Trade-offs & defeaters
14. Memorization card

1. Crux & why hard

- Naive: clients POST each metric point to an API that inserts into a DB → network drowns in tiny requests, DB dies on write QPS, queries compete with writes, one slow consumer stalls everything.
- Minimum primitives: **agent-side batching**, **a partitioned log as shock absorber** (Kafka, sd15), **write/query separation** (TSDB + query tier), **at-least-once + idempotent sinks**.
- 🗨️ "The write path is a firehose of tiny appends, the read path is aggregation over ranges — the whole design is keeping those two from touching."

2. Clarifying questions (assumed answers)

- Scale? → $100k \text{ hosts} \times 100 \text{ series} \times 1 \text{ pt}/10 \text{ s} \approx 1M \text{ points/s}$
- Loss tolerance? → metrics: seconds of loss OK on agent crash; healthcare variant: zero loss (\$12)
- Query pattern? → dashboards: recent ranges, aggregated; rarely raw old data
- Cardinality bound? → must cap labels (the classic TSDB killer)

3. FR / NFR

Req	Target	Enforced by
Ingest	1M pts/s sustained	agent batching + Kafka partitions
E2E freshness	< 30 s to queryable	consumer lag alarms
Availability	ingest survives TSDB outage	buffer in log (retention 24h+)
Delivery	at-least-once, dedup at sink	offsets + idempotent writes
Query	p99 < 2 s dashboard range	pre-aggregation + separate query tier

4. Back-of-envelope

- $1\text{M pts/s} \times \sim 50\text{ B} \approx 50\text{ MB/s}$ raw — trivially Kafka-sized (one modest cluster) — so what: broker is not the bottleneck, cardinality and TSDB write amp are
- Agent batching $10\text{ s} \rightarrow 100\text{k hosts} \times 1\text{ req}/10\text{ s} = 10\text{k rps}$ at the gateway instead of 1M — so what: the agent IS the scalability feature

5. HLD

```
[hosts] agent (aggregate 10s, batch, disk buffer)
  | HTTPS batch [hot path]
  ▼
gateway (authn, per-tenant rate limit sd01, validate)
  | produce, key = hash(tenant, metric)
  ▼
Kafka topic (N partitions, RF=3, acks=all)      [shock absorber]
  ├── ingest consumers → TSDB (time-partitioned) [async]
  ├── rollup stream (1m/1h pre-aggregates)      [async]
  └── DLQ topic (poison records)                  [async]
query tier → TSDB + rollups (never touches ingest path)
[control plane] schema/limits config → gateway + consumers
```

6. Client-side agent (key code) — the named discussion point

- Aggregate locally (10 s windows), batch, compress; spill to bounded disk buffer when gateway unreachable; jittered flush; drop-oldest beyond buffer cap (metrics) — say the loss window out loud.

```

import time, threading

class Agent:
    def __init__(self, sink, flush_s=10, max_buf=10_000):
        self.acc = {} # (name, labels) -> [count, total, mx]
        self.backlog = [] # failed batches, bounded
        self.sink, self.flush_s, self.max_buf = sink, flush_s, max_buf
        self.lock = threading.Lock()

    def record(self, name, labels, value):
        with self.lock:
            c = self.acc.setdefault((name, labels), [0, 0.0, float('-inf')])
            c[0] += 1; c[1] += value; c[2] = max(c[2], value)

    def flush(self): # called every flush_s (+ jitter)
        with self.lock:
            batch, self.acc = self.acc, {}
            payload = [(k, tuple(v), int(time.time())) for k, v in batch.items()]
            try:
                self.sink.send(self.backlog + payload)
                self.backlog = []
            except Exception:
                self.backlog = (self.backlog + payload)[-self.max_buf:] # drop oldest
# pre-aggregation turns 1M pts/s into 10k rps; bounded backlog = stated loss window

```

```

class Agent {
    private final Map<SeriesKey, Agg> acc = new ConcurrentHashMap<>();
    private final Deque<Batch> backlog = new ArrayDeque<>(); // bounded

    void record(SeriesKey k, double v) {
        acc.compute(k, (key, a) -> (a == null ? new Agg() : a).add(v));
    }

    void flush(Sink sink) { // scheduled every 10s with jitter
        Map<SeriesKey, Agg> batch = new HashMap<>(acc);
        acc.keySet().removeAll(batch.keySet());
        try { sink.send(backlog, batch); backlog.clear(); }
        catch (Exception e) {
            backlog.addLast(Batch.of(batch));
            while (backlog.size() > MAX_BUF) backlog.pollFirst(); // drop oldest
        }
    }
}
// compute() keeps record() lock-free-ish on the hot path

```

7. Sharding & broker choice

- Partition key `hash(tenant, metric_name)` → same series ordered in one partition; hot-tenant skew → add sub-key (metric) or salted keys
- Broker: Kafka over SQS — need replay, ordering per key, many independent consumers (TSDB, rollups, alerting); SQS = work queue, no replay (sd15 has the mirror argument)
- Consumers in a group, one partition each → scale by partitions; lag = the health metric

8. Write path vs query path — the named discussion point

- Writers append; TSDB partitioned by time (e.g. 2 h blocks), recent block in memory
- Rollup consumers write 1m/1h aggregates — dashboards read rollups, raw only for zoom-in
- Query tier is separate deployment: a bad dashboard query can never slow ingest
- Retention tiers: raw 15 d, 1m × 6 mo, 1h × 2 y (cost story)

9. Data model

```
-- TSDB logical layout (Prometheus/M3-style)
-- series identified once, points reference series_id (dedup label bytes)
CREATE TABLE series (
  series_id  BIGINT PRIMARY KEY,          -- hash(tenant, name, sorted_labels)
  tenant_id  TEXT NOT NULL,
  metric_name TEXT NOT NULL,
  labels     JSONB NOT NULL
);
CREATE TABLE points_2h (                -- one physical partition per 2h window
  series_id BIGINT,
  ts         TIMESTAMPTZ,
  cnt BIGINT, sum DOUBLE PRECISION, max DOUBLE PRECISION,
  PRIMARY KEY (series_id, ts)
);
-- Kafka: topic metrics.{env}, key=hash(tenant,metric), value=proto batch
-- DLQ:  topic metrics.dlq, headers carry error + source offset for replay
```

10. Failure handling: DLQ + replay

- Poison record (bad schema, oversized) → after N parse attempts, produce to DLQ with error header; never block the partition
- Sink outage → consumers pause, log retains (24 h+ buffer); catch-up is just lag draining
- Replay tool: re-produce DLQ range back into main topic after fix; idempotent sink (upsert by `series_id, ts`) makes replay safe
- ? "Exactly once?" → at-least-once + idempotent upsert ≈ effectively-once at the sink; say it in those words

11. Ops readiness

- Dashboards: gateway rps/4xx, consumer lag per partition, DLQ rate, TSDB block flush time

- Alarms: lag > 5 min, DLQ > 0.1%, cardinality growth per tenant (limit + reject with clear error)
- Runbook: hot partition → check key skew; TSDB down → verify retention window covers expected outage

12. Healthcare records variant (bank #5 — same skeleton, different dials)

- Zero loss: agent disk buffer unbounded-ish + backpressure instead of drop-oldest; gateway acks only after Kafka `acks=all`
- Compliance: PHI encrypted at rest + in transit; per-facility partition key; audit every access (sd13's audit thinking); retention = years, raw kept in a lake (Parquet) + structured store for queries
- Ordering: per-patient ordering via partition key `hash(facility, patient)`
- Failed/unprocessed records — an explicitly named discussion point: DLQ + operator replay UI + daily reconciliation count (produced vs landed)
- 🧭 walk the 8 named points in order: ingestion → broker choice → storage → processing → fault tolerance → scalability → monitoring/alerting → failed-record handling; the question IS the outline

13. Trade-offs & defeaters

- Defeater: "agent sends every point live" — 本地聚合 + 批量, otherwise the network dies first
- Defeater: "write straight to the DB, skip the log" — a TSDB hiccup then drops data; the log decouples availability
- Defeater: "at-most-once for throughput" (healthcare) — losing medical records is an incident; at-least-once + idempotent consumers
- Kafka vs Kinesis/SQS: replay + multi-consumer + per-key order → log; simple task fan-out → queue

14. Memorization card

- Agent batches 10 s (1M pts/s → 10k rps) with bounded spill; Kafka (key = tenant+metric, acks=all) absorbs; consumers → TSDB + rollups; query tier separate
- At-least-once end-to-end + idempotent upsert; DLQ + replay for poison
- Healthcare dials: no drop, backpressure, PHI encryption, audit, per-patient ordering, reconciliation
- Watch: consumer lag, DLQ rate, cardinality